pyAmpliCol at one loop

The recursive one-loop engine against MadLoop, Recola, OpenLoops and NJet

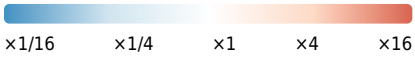
2026-10-24 · values from `loop`, G1 at `4e9b5a9e` and `work/v2-generation-scale` (before its integration into `loop` as G-int at `4e9b5a9e`): validation records up to `6f5a64cc` · engine `4e9b5a9e` on `loop` · AMD EPYC 9754, shared host · binary64 · one pinned thread · batches of $B = 128$

Reading guide. One row per test process, by multiplicity. A cell gives pyAmpliCol's value in bold and, at its right, the multiplier *pyAmpliCol ÷ the fastest provider* on this host (in bold among the provider values); the colour encodes it (blue faster, red slower). Every value is the **most recent measured configuration** of its cell (by git ancestry of the revisions; a newer check that only repeats part of an older comparison does not replace it): $2 \to 2$ to $2 \to 4$: the engine on `loop` with native per-point inputs, gate G1 at `4e9b5a9e` (2026-10-24); generation and validation of the $2 \to 5$ rows: the validation records of `work/v2-generation-scale` up to `6f5a64cc` (fixture points, shared host under load, no batched timing), made before its integration into `loop` (G-int, `4e9b5a9e`). The values of $2 \to 5$ are **development measurements**, not yet repeated on `loop`. The engine before G-int (Python per-point inputs, G1 at `1c971289`) is history in Section 2. Grey cells say why a value is missing. Run times: per phase-space point in batches, finite part, stability checks off; providers in production mode (Section 1).

Table 1: **State of the one-loop engine.** Every value from `data/summary.json` (`scripts/build_summary.py`).

	Process · validation	Generation	Full colour · per point, batched	Leading colour (LC)
	pyAmpliCol values: the most recent measured configuration · $2 \to 2$ to $2 \to 4$: `loop` `4e9b5a9e`, native per-point inputs, G1 2026-10-24 · generation and validation of $2 \to 5$: `work/v2-generation-scale` validation records up to `6f5a64cc` (before G-int)			
	legs · corrections · loop families; largest deviation of the finite part per reference	generate + plan export, peak memory, plan size; provider generation on this host	median per point at the largest batch; provider times; gate G1 (time ÷ limit); stage shares	batched, same configuration; LC/FC; no provider ratio
$2 \to 2$	$gg \to gg$ 4 legs · QCD · 24 families (LC 12) Rc·OL·ML ≤ 1.7e−12 16 pts · median 1.5e−14	**10.1 s** ×71 0.30 GiB peak · plan 0.305 MB **Rc 142 ms** · ML 9.70 s	**2.21 ms** ×1.1 **ML 2.02 ms** · OL 3.20 ms · Rc 10.0 ms $B = 128$ no gate (2→2) untimed 48 %	**1.56 ms** LC/FC 0.71 $B = 128$ · FC 2.21 ms
$2 \to 3$	$u\bar{u} \to d\bar{d}g$ 5 legs · QCD · 42 families (LC 27) OL·ML ≤ 6.8e−13 · Rc 3.3e−10 16 pts · median 8.1e−14	**10.5 s** ×26 0.30 GiB peak · plan 0.282 MB **Rc 406 ms** · ML 9.87 s	**2.67 ms** ×1.8 **OL 1.52 ms** · ML 3.17 ms · Rc 3.18 ms $B = 128$ fail ×1.4 untimed 40 %	**1.89 ms** LC/FC 0.71 $B = 128$ · FC 2.67 ms
	$u\bar{u} \to u\bar{u}g$ 5 legs · QCD · 68 families (LC 42) ML·OL ≤ 6.8e−12 · Rc 7.2e−11 16 pts · median 1.3e−13	**10.7 s** ×23 0.31 GiB peak · plan 0.388 MB **Rc 462 ms** · ML 10.9 s	**3.30 ms** ×1.1 **OL 2.92 ms** · Rc 5.55 ms · ML 8.59 ms $B = 128$ pass ×0.87 numerator 37 %	**2.72 ms** LC/FC 0.82 $B = 128$ · FC 3.30 ms
	$u\bar{u} \to e^-e^+g$ 5 legs · QCD · 11 families (LC 7) Rc·ML ≤ 8.8e−14 16 pts · median 8.3e−15 · †	**26.2 s** ×37 0.33 GiB peak · plan 0.310 MB **Rc 701 ms** · ML 14.0 s	**0.796 ms** ×0.84 **ML 0.948 ms** · Rc 1.08 ms · OL no library $B = 128$ pass ×0.84 numerator 34 %	**0.622 ms** LC/FC 0.78 $B = 128$ · FC 0.796 ms
	$u\bar{u} \to ggg$ 5 legs · QCD · 90 families (LC 62) ML·Rc ≤ 1.5e−12 · OL 8.4e−11 16 pts · median 4.4e−14	**11.8 s** ×19 0.33 GiB peak · plan 0.656 MB **Rc 634 ms** · ML 17.6 s	**10.2 ms** ×0.82 **OL 12.4 ms** · Rc 25.3 ms · ML 28.2 ms $B = 128$ pass ×0.63 untimed 59 %	**12.0 ms** LC/FC 1.18 $B = 128$ · FC 10.2 ms
	$gg \to ggg$ 5 legs · QCD · 154 families (LC 77) ML·Rc·OL ≤ 1.7e−6 16 pts · median 3.2e−13 · †	**20.0 s** ×3.0 0.44 GiB peak · plan 1.85 MB **Rc 6.63 s** · ML 47.0 s	**45.4 ms** ×0.47 **OL 97.0 ms** · ML 247 ms · Rc 351 ms $B = 128$ pass ×0.36 untimed 67 %	**44.7 ms** LC/FC 0.98 $B = 128$ · FC 45.4 ms
$2 \to 4$	$u\bar{u} \to e^-e^+gg$ 6 legs · QCD · 69 families (LC 33) Rc·ML ≤ 1.1e−10 16 pts · median 6.4e−14 · †	**27.9 s** ×33 0.34 GiB peak · plan 0.624 MB **Rc 833 ms** · ML 24.8 s	**6.23 ms** ×0.61 **Rc 10.2 ms** · ML 27.3 ms · OL no library $B = 128$ pass ×0.47 numerator 44 %	**3.42 ms** LC/FC 0.55 $B = 128$ · FC 6.23 ms
	$gg \to t\bar{t}gg$ 6 legs · QCD · 1 086 families (LC 375) OL·Rc·ML ≤ 7.6e−9 16 pts · median 2.0e−11 · †	**35.7 s** ×0.58 0.66 GiB peak · plan 3.71 MB **Rc 61.4 s** · ML 218 s	**183 ms** ×0.39 **OL 472 ms** · Rc 1.97 s · ML 5.45 s $B = 128$ pass ×0.30 untimed 59 %	**217 ms** LC/FC 1.18 $B = 128$ · FC 183 ms
	$u\bar{u} \to gggg$ 6 legs · QCD · 628 families (LC 375) Rc 9.4e−12 · OL 1.9e−10 16 pts · median 1.5e−12 · †	**37.3 s** ×1.1 0.72 GiB peak · plan 4.54 MB **Rc 32.8 s**	**170 ms** ×0.46 **OL 374 ms** · Rc 908 ms · ML not exported $B = 128$ pass ×0.35 untimed 63 %	**218 ms** LC/FC 1.28 $B = 128$ · FC 170 ms
	$gg \to gggg$ 6 legs · QCD · 1 100 families (LC 550) ML·Rc·OL ≤ 4.0e−7 8 pts · median 3.1e−11 · †	**148 s** ×0.070 2.15 GiB peak · plan 16.1 MB **Rc 35.4 min** · ML 144.6 min	**913 ms** ×0.32 **OL 2.84 s** · Rc 17.6 s · ML 38.7 s $B = 128$ pass ×0.25 untimed 71 %	**1.03 s** LC/FC 1.13 $B = 128$ · FC 913 ms · M (39824f9b): 1.02 s

	Process · validation	Generation	Full colour · per point, batched	Leading colour (LC)
pyAmpliCol values: the most recent measured configuration · $2 \to 2$ to $2 \to 4$: `loop` 4e9b5a9e, native per-point inputs, G1 2026-10-24 · generation and validation of $2 \to 5$: `work/v2-generation-scale` validation records up to `6f5a64cc` (before G-int)				
	legs · corrections · loop families; largest deviation of the finite part per reference	generate + plan export, peak memory, plan size; provider generation on this host	median per point at the largest batch; provider times; gate G1 (time ÷ limit); stage shares	batched, same configuration; LC/FC; no provider ratio
$2 \to 5$	$u\bar{u} \to e^-e^+d\bar{d}g$ 7 legs · QCD · families not recorded ML·Rc·OL ≤ 8.7e−13 3 pts · †	**37.7 s** ×4.5 0.41 GiB peak · plan 1.13 MB **Rc 8.40 s** · ML 55.7 s	***batched timing pending (measure-g1-7leg)*** production, s/pt (20 Oct): OL 0.053–0.055 · Rc 0.035 · ML 0.143 gate limit 0.0455 s/pt (Recola)	***batched timing pending (measure-g1-7leg)***
	$u\bar{u} \to e^-e^+ggg$ 7 legs · QCD · families not recorded OL·ML·Rc ≤ 8.5e−13 3 pts	**36.7 s** ×2.3 0.48 GiB peak · plan 2.45 MB **Rc 15.9 s** · ML 142 s	***batched timing pending (measure-g1-7leg)*** production, s/pt (20 Oct): OL 0.348–0.353 · Rc 0.246–0.259 · ML 1.41 gate limit 0.3198 s/pt (Recola)	***batched timing pending (measure-g1-7leg)***
	$gg \to t\bar{t}e^-e^+g$ 7 legs · QCD · families not recorded Rc·ML·OL ≤ 5.7e−9 3 pts	**43.7 s** ×1.5 0.59 GiB peak · plan 2.13 MB **Rc 28.7 s** · ML 161 s	***batched timing pending (measure-g1-7leg)*** production, s/pt (20 Oct): OL 0.425–0.433 · Rc 0.59–0.67 · ML 3.03–3.04 gate limit 0.5525 s/pt (OpenLoops)	***batched timing pending (measure-g1-7leg)***
	$gg \to t\bar{t}e^-e^+g$ 7 legs · EW · families not recorded **not started (WP3.2): EW loops**	***not started (WP3.2)*** EW loops: no configuration generates it yet **Rc 21.9 s** · ML 10.9 min	***not started (WP3.2)*** production, s/pt (20 Oct): OL 0.96–1.12 · Rc 0.95–1.02 · ML 6.90–7.08 gate limit 1.235 s/pt (Recola)	***not started (WP3.2)***
	$u\bar{u} \to ggggg$ 7 legs · QCD · families not recorded ML 2.2e−12 · Rc 2.9e−10 3 pts	**517 s** ×0.051 6.10 GiB peak · plan 69.2 MB **Rc 170.3 min** · ML 12.2 h	***batched timing pending (measure-g1-7leg)*** production, s/pt (20 Oct): OL no library · Rc not measured (protocol run, poles on, cache off: 180–184) · ML 569–570 no complete gate limit (Recola production not measured)	***batched timing pending (measure-g1-7leg)***
	$gg \to ggggg$ 7 legs · QCD · families not recorded (9 414 with top loops) NJet DD 3.8e−9 record `0ea83af2` · 3 pts · † · top-free (particle_veto = [t]), Nf = 5	**51.5 min** 28.7 GiB peak · plan 160 MB ML diagrams 17.4 h, stopped in export · Rc unfinished	***batched timing pending (measure-g1-7leg)*** NJet 3.1.2 71.2 s per evaluation (one core; not matched) no matched provider: reported without a gate (addendum H)	***batched timing pending (measure-g1-7leg)***



RdBu, $\log_2$ scale, clamped. Grey: missing (reason given). White without a multiplier: no provider ratio (leading colour).

Stage shares (bar; the largest is named): inputs numerator scatter NinjaRS masters glue untimed (Table 4): inputs = trees, Born and counterterm inputs; glue = point parameters, input conversion, assembly; untimed = rest of the call (on `loop`: native inputs, Python rest). Times use the protocol's schedule B; "M:" gives the schedule G1 adopted for the class (Table 9; its run's revision where not repeated).
References: ML MadLoop 3.7.0, Rc Recola 2.2.5 (COLLIER), OL OpenLoops 2.1.5 (prebuilt libraries: no generation time), NJet DD NJet 3.1.2 in double-double. Deviation $|a-b| \,/\, \max(|a|,|b|,|B|\ \alpha_s/2\pi)$ of the binary64 finite part at the timed points (median: over the points, the largest over the references); † the largest lies within the binary64 stability estimate of the same configuration at that point (G1 rows: stability runs of `39824f9b`, bitwise the same binary64 values). **Gate chip:** time ÷ binding limit (MadLoop, or 1.30 × the faster of Recola and OpenLoops); pass or fail when the whole uncertainty band is below or above 1, else open.

Configurations (main-configuration rule of `scripts/build_summary.py`; `data/summary.json` names the configuration and source of every value). $2 \to 2$ to $2 \to 4$: the engine on `loop`, gate G1 at `4e9b5a9e`, build `loop-native-v2-slice-4e9b5a9e`, native per-point inputs (Section 2). Generation and validation of the $2 \to 5$ rows: the validations of `work/v2-generation-scale` with native inputs and the production counterterm route (DESIGN WP3.1, ninth, eighth and seventh increments; Table 12), run on the shared host under its load, not on the idle pinned cores of the G1 protocol. Partial repeats leave the more complete comparison in the table: $gg \to ggggg$ (ninth, 1 of 3 points) the eighth increment's (`0ea83af2`). G1 has not timed the $2 \to 5$ rows yet (`measure-g1-7leg`): their gate limits come from the providers' production times (20 Oct, DESIGN §1.3 Table 1b). Generation excludes model loading; unbatched development timings (e.g. a seven-gluon point with its rotation check) are not in this table (Section 4).

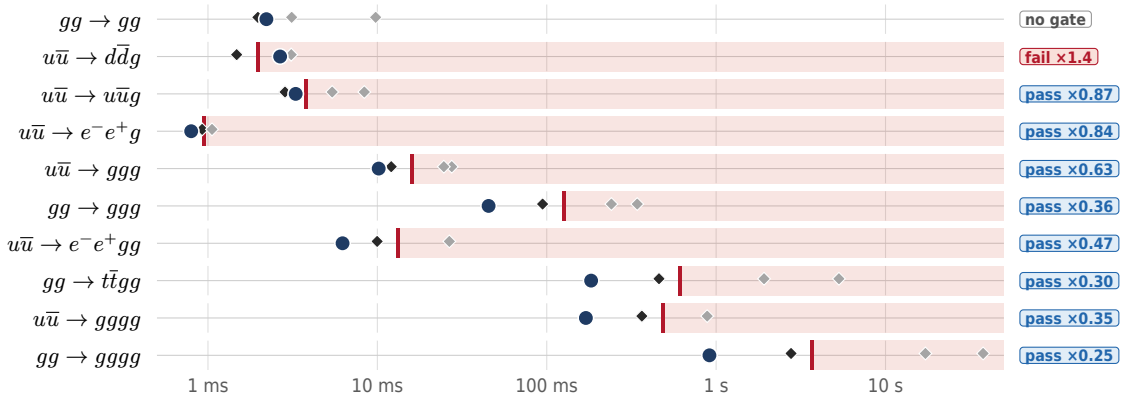


Figure 1: **Batched full colour against the gate** ($B = 128$ or the largest batch, finite part; the rows of Table 1 with a matched provider). Dots: pyAmpliCol's values of Table 1, the most recent measured configuration ($2 \to 2$ to $2 \to 4$: `loop` `4e9b5a9e`, native per-point inputs, G1 2026-10-24); diamonds: the providers, the darkest the fastest; red line: the gate limit, the shaded side fails. The ratios and bands at every batch size: Table 3.

1 Status at this milestone

On loop (engine 4e9b5a9e; merges since c3a7be09, engine b4d76cc3)

c29d5899	Rust A3	**Native counterterm-program builder.**
4e9b5a9e	WP3.1 G-int	**Native per-point inputs and generation stages.**

Before (engine b4d76cc3): v2 waves 0-1, gates S1 and G0, native executor (WP2.1), colour (WP2.4), stability API (WP2.3), schedule M (WP2.8), tree-EW follow-up, NJet reference, observables (WP2.5), public v2 engine with plan v4 artifacts (WP2.6), counterterms in the recursion (WP2.2), runtime pole selection (WP2.10), Rust A0 (contract-class reuse pass), constant counterterm arguments (ct-assume-fix), addendum F-J, Rust A1-A2 (native loop-request DAG builder), the gate G1 harness (WP2.7), counterterms as plain tree insertions (addendum J, b4d76cc3), the summary-first report. **Native build** loop-native-v2-slice-4e9b5a9e (digest 02e1e95c…).

Gate G1 (Section 2; finite part, $2 \to 3$ and beyond)

- **Most recent configuration** (Table 1 and Figure 1): native per-point inputs on loop (G1 at 4e9b5a9e) at $B = 128$: 8 of 9 pass, $u\bar{u} \to d\bar{d}g$ fails (×1.35 of the limit). At $B = 1$ no process passes with 16 lanes (6 with one lane: the development runs of 39824f9b, history; the public path runs 16 lanes).
- **Before G-int** (Python per-point inputs, G1 at 1c971289; history, Section 2): failed every gated process it generated, at ×11–20 of the binding limit at $B = 128$ (the Python inputs took 87 %–99 % of the time) and at ×12–27 for $gg \to gggg$, $gg \to t\bar{t}gg$ and $u\bar{u} \to gggg$, timed at $B = 1$ only, and refused $u\bar{u} \to e^-e^+g$ and $u\bar{u} \to e^-e^+gg$.
- **$2 \to 5$:** not timed yet (measure-g1-7leg); $u\bar{u} \to ggggg$ has no complete gate limit (Recola production not measured); $gg \to ggggg$ has no matched provider and is reported without a gate.

In development (branches, not on loop; HANDOFF §6)

- **Counterterm-route parity** (ct-perf, with Rust A4): output-identical Python gains, then A4 switches the counterterm program and the loop-block DAG to the native builder of A3. The pure-gluon generation on loop is slower than on the development branch before the merge ($gg \to gggg$: generate 116 s, 64.5 s at 39824f9b in the previous campaign), consistent with the counterterm-route excess ct-perf addresses (not attributed stage by stage).
- **Rust ports** (HANDOFF §0.4): A0, A1–A2 and A3 are on loop, not switched on; seeds (B) and cells (C) passed phase 0, lowering (D) is in a fix round; their switches, A5 and A6 wait for counterterm-route parity.
- **Next, after parity:** generation-scale-2 (every stored-process call without regeneration, SDK v4 runtime, native Catani operator, lean poles companion); artifact portability (branch); the seven-leg G1 rows (measure-g1-7leg, specifications and streams).

Missing or incomplete

- Electroweak loops, Goldstones and the complex-mass scheme (WP3.2): $gg \to t\bar{t}e^-e^+g$ (EW) not started. (Z couplings on loop quark lines in QCD loops generate on loop since G-int; the Python-input engine refused them.)
- Gate G1 at $B = 1$ (16 lanes for one point) and for $u\bar{u} \to d\bar{d}g$ at $B = 128$; batched timings of the $2 \to 5$ set; provider protocol-mode columns.
- Seven-gluon generation cost (WP3.1 branch, before G-int): 51.5 min with the plan export (generation 44.2 min, mostly the always-generated poles companion), peak 28.7 GiB (Section 4); inside addendum H, to be made lean.
- Ward check at seven gluons. Prepared artifacts (addendum G): a plain evaluation of a stored process loads without regeneration since G-int, but correlated, resolved and flavour calls, the "uv"/"ir" selections and the writer's regenerated-at-load fallback still regenerate; the native build embeds host paths of its build.

Performance gates and timing protocol: definitions

- **Gate.** Faster than MadLoop and at most 1.30× each matched Recola and OpenLoops time, on the **finite part**, from $2 \to 3$ upward; $2 \to 2$ is reported without a gate. Aspiration: fastest of all tools from $2 \to 3$.
- **Protocol.** Providers in production mode with poles off where they allow it (OpenLoops truepoles 0, Recola without IR poles, MadLoop production build); pyAmpliCol finite-only (poles = "none") with stability checks off, one pinned thread, binary64, 16 lanes, the same frozen points (pyAmpliCol receives their exact rationals, the providers their binary64 decimal text). Statistic: median per point; the uncertainty band combines pyAmpliCol's block spread and the provider's repeat spread; a gate whose band contains the limit is **open**.
- **Timing boundary.** pyAmpliCol: the public v2 runtime call (NativeExecutor.evaluate) for a batch of B distinct points, everything included (per-point inputs, loop stage, assembly); the public Runtime.evaluate of a loaded artifact adds its input parsing (Section 2). Providers have no batch API: one time per point serves every B. Generation: pyAmpliCol LoopSlice.generate plus the plan export, in G1 on one idle pinned core; a development record's generation in Table 1 ran on the shared host under load (threads per increment: Table 12), without a multiplier in a G1 row; MadLoop graph generation, export and compilation; Recola process generation; OpenLoops libraries are prebuilt (no generation time).
- **Labels.** Table 1 and Figure 1 show the most recent measured configuration of each cell (main-configuration rule of scripts/build_summary.py; data/summary.json names the configuration and source of every value); their title line, reading guide and column header state it once, with branch and commit, and say when it is a development measurement not yet on loop. The detailed G1 tables (Section 2) show the configuration on loop and, labelled, its history: the Python per-point inputs before G-int and the development runs of 39824f9b. Missing values are marked, never estimated or extrapolated; test durations are never used as event timings. The legacy per-diagram engine's timings of 18 October are history (data kept in data/measurements.json, not shown).

2 Gate G1: end-to-end timings of the v2 engine

G1 times the complete Born–virtual evaluation, **finite part only**, through the call of the public v2 runtime (NativeExecutor.evaluate, which V2VirtualRuntime calls) with stability checks off, on one pinned core and one thread,

binary64, 16 SIMD lanes, in batches of B distinct points of a frozen stream (REQUIREMENTS_ADDENDUM D, plan §5.8); pyAmpliCol receives the stream's exact rational points, the providers their binary64 decimal text. The configurations:

- **native inputs, on `loop`** — the engine at `4e9b5a9e` (G-int; build `loop-native-v2-slice-4e9b5a9e`): the executor forms the trees, the Born and the counterterm inputs per point, then runs the loop stage. A stored artifact written by the public generator holds the plan v4 with `point_inputs = "native-v1"` and loads without any generation call; for $u\bar{u} \to d\bar{d}g$ it has the harness's plan and, on the exact points, bitwise its values at all 128 stream points (the plan also checked for $u\bar{u} \to u\bar{u}g$). It generates the V+jets rows with γ and Z exchange. Its binary64 values equal, bitwise at every timed point in full and leading colour, those of the development campaign of `39824f9b` (`work/v2-generation-scale` `889af02c` plus timers), whose runs are its earlier runs; that campaign's one-lane, schedule-M and stability runs were not repeated and appear below as history, labelled `39824f9b`;
- **Python inputs** history — the engine on `loop` before G-int, G1 at `1c971289` (LC at `d591a2f7`): trees, Born amplitudes and counterterm scalars formed per point in Python, then the loop stage natively. It refused the V+jets rows: their closed-quark Agg triangles needed Furry's certified zeros (WP3.5), and their Z couplings on loop quark lines the electroweak fermion signs (WP3.2).

Table 2: **Gate G1, full colour**, ms per point (gate statistic: median), finite part, one thread. Rows $B = 1$ and $B = 128$ (the Python-input $2 \to 4$ rows only at $B = 1$: their Python inputs took seconds per point). Multiplier and colour: time ÷ the fastest provider of the row (legend of Table 1). Grey rows: generation — pyAmpliCol generation plus plan export with its peak memory; MadLoop graph generation, export and compilation · the share of its loop-numerator coefficients in its time (Table 4); Recola process generation; OpenLoops: the prebuilt library and its load plus first call. MadLoop $u\bar{u} \to gggg$ was not exported: the $gg \to gggg$ export took 2.4 h here (HANDOFF §0.1), and OpenLoops binds the gate. Gate: upper line Python inputs (history), lower line native inputs (`loop`); the number is the time over the binding limit (MadLoop's time or 1.30× the faster of Recola and OpenLoops); **open**: the band contains the limit (bands: Table 3).

Process	B	Python inputs[hist]	Native inputs, `loop`	MadLoop	Recola	OpenLoops	Gate
$gg \to gg$ $2 \to 2$, no gate	1	**140** ×69	**14.3** ×7.1	**2.02** fastest	**10.0** ×5.0	**3.20** ×1.6	no gate
	128	**132** ×66	**2.21** ×1.1	**2.02** fastest	**10.0** ×5.0	**3.20** ×1.6	no gate
	gen.	10.4 s · 0.3 GiB	10.1 s · 0.3 GiB	9.70 s · 64 %	0.142 s	ppjj · 6.80 ms	
$u\bar{u} \to d\bar{d}g$ $2 \to 3$	1	**40.6** ×27	**10.9** ×7.2	**3.17** ×2.1	**3.18** ×2.1	**1.52** fastest	**fail** ×21 **fail** ×5.5
	128	**32.4** ×21	**2.67** ×1.8	**3.17** ×2.1	**3.18** ×2.1	**1.52** fastest	**fail** ×16 **fail** ×1.4
	gen.	9.94 s · 0.3 GiB	10.5 s · 0.3 GiB	9.87 s · 73 %	0.406 s	ppjjj · 4.73 ms	
$u\bar{u} \to u\bar{u}g$ $2 \to 3$	1	**68.9** ×24	**18.9** ×6.5	**8.59** ×2.9	**5.55** ×1.9	**2.92** fastest	**fail** ×18 **fail** ×5.0
	128	**54.5** ×19	**3.30** ×1.1	**8.59** ×2.9	**5.55** ×1.9	**2.92** fastest	**fail** ×14 **pass** 0.87
	gen.	10.4 s · 0.3 GiB	10.7 s · 0.3 GiB	10.9 s · 75 %	0.462 s	ppjjj · 8.43 ms	
$u\bar{u} \to e^-e^+g$ $2 \to 3$	1	*not generable*	**3.65** ×3.8	**0.948** fastest	**1.08** ×1.1	*no library*	— **fail** ×3.8
	128	*not generable*	**0.796** ×0.84	**0.948** fastest	**1.08** ×1.1	*no library*	— **pass** 0.84
	gen.	CT coverage refused	26.2 s · 0.3 GiB	14.0 s · 79 %	0.701 s	—	
$u\bar{u} \to ggg$ $2 \to 3$	1	**215** ×17	**49.2** ×4.0	**28.2** ×2.3	**25.3** ×2.0	**12.4** fastest	**fail** ×13 **fail** ×3.1
	128	**179** ×14	**10.2** ×0.82	**28.2** ×2.3	**25.3** ×2.0	**12.4** fastest	**fail** ×11 **pass** 0.63
	gen.	12.3 s · 0.4 GiB	11.8 s · 0.3 GiB	17.6 s · 76 %	0.634 s	ppjjj · 23.7 ms	
$gg \to ggg$ $2 \to 3$	1	**2 760** ×28	**209** ×2.2	**247** ×2.5	**351** ×3.6	**97.0** fastest	**fail** ×22 **fail** ×1.7
	128	**2 560** ×26	**45.4** ×0.47	**247** ×2.5	**351** ×3.6	**97.0** fastest	**fail** ×20 **pass** 0.36
	gen.	27.9 s · 0.6 GiB	20.0 s · 0.4 GiB	47.0 s · 78 %	6.63 s	ppjjj · 159 ms	
$u\bar{u} \to e^-e^+gg$ $2 \to 4$	1	*not generable*	**42.7** ×4.2	**27.3** ×2.7	**10.2** fastest	*no library*	— **fail** ×3.2
	128	*not generable*	**6.23** ×0.61	**27.3** ×2.7	**10.2** fastest	*no library*	— **pass** 0.47
	gen.	CT coverage refused	27.9 s · 0.3 GiB	24.8 s · 83 %	0.833 s	—	
$gg \to gggg$ $2 \to 4$	1	**101 000** ×36	**4 000** ×1.4	**38 700** ×14	**17 600** ×6.2	**2 840** fastest	**fail** ×27 **fail** ×1.1
	128	*not timed*	**913** ×0.32	**38 700** ×14	**17 600** ×6.2	**2 840** fastest	— **pass** 0.25
	gen.	335 s · 4.1 GiB	148 s · 2.1 GiB	8 680 s · 51 %	2 130 s	ppjjjj · 4 950 ms	
$gg \to t\bar{t}gg$ $2 \to 4$	1	**7 310** ×15	**829** ×1.8	**5 450** ×12	**1 970** ×4.2	**472** fastest	**fail** ×12 **fail** ×1.4
	128	*not timed*	**183** ×0.39	**5 450** ×12	**1 970** ×4.2	**472** fastest	— **pass** 0.30
	gen.	62.5 s · 0.9 GiB	35.7 s · 0.7 GiB	218 s · 64 %	61.4 s	pptjj · 821 ms	

Process	B	Python inputs[hist]	Native inputs, `loop`	MadLoop	Recola	OpenLoops	Gate
$u\bar{u} \to gggg$ $2 \to 4$	1	**5 760** ×15	**845** ×2.3	*not timed*	**908** ×2.4	**374** fastest	**fail** ×12 **fail** ×1.7
	128	*not timed*	**170** ×0.46	*not timed*	**908** ×2.4	**374** fastest	— **pass** 0.35
	gen.	56.1 s · 0.7 GiB	37.3 s · 0.7 GiB	—	32.8 s	ppjjj · 606 ms	

The providers run in production mode on the same stream: OpenLoops with `truepoles 0`, Recola without IR poles (one amplitude call per point), MadLoop at its production stability setting on exports built with MG5′s production flags. OpenLoops has no installed library with QCD corrections to $\ell^+\ell^-$ + 1 or 2 jets (`pplj_ew` carries electroweak corrections only; existing provider builds only), so the V+jets gates bind on MadLoop and Recola. Providers have no batch API: one time per point serves every B. **Gate statistic:** the median per point on both sides — over pyAmpliCol's timed blocks of one B, and over the provider's timed points (the stream's first 16; 8 for MadLoop $gg \to gggg$, Recola $gg \to gggg$, OpenLoops $gg \to gggg$ and Recola $u\bar{u} \to gggg$). The provider times are the records of the measure-g1 campaign (23–24 October, on cores 129–250), not re-run for this milestone; pyAmpliCol ran on cores 131–178, one job per idle eight-core group sharing an L3 cache. Its uncertainty is pyAmpliCol's block spread (half the range of three blocks, the median's standard error from five) plus the provider's half-range over its repeated runs (8 %, the campaign's largest, for the single runs of Recola $gg \to gggg$ and Recola $u\bar{u} \to gggg$). A gate passes or fails only if the whole band does; otherwise it is **open** and the configuration is repeated once (plan §5.8 item 10). Every timed call returned all its points (failed points: 0); rescue is off by protocol (no checks, no escalation), so the rescue rate is 0.

Table 3: **The gate at every batch size** (native inputs on `loop`, full colour): time over the binding limit and its uncertainty band (cell colour: blue pass, red fail, amber open). $B = 1$ is timed with the default 16 lanes (a one-point call pays a 16-lane chunk); the one-lane column is the development campaign's (`39824f9b`, history: the public path always runs 16 lanes in binary64). This campaign timed $B = 1$ and $B = 128$, and every batch size only for the single repeat of $u\bar{u} \to u\bar{u}g$ (its first run, 3.78 ms at $B = 128$, was open at the limit: `repeats` in `data/g1.json`); ° grey: the earlier run of `39824f9b` at that batch size, its median over the same binding limit with its time (no band; bitwise the same values; not a verdict of this campaign).

Process	$B = 1$, 16 lanes		$B = 1$, 1 lane (39824f9b)		$B = 8$		$B = 32$		$B = 128$	
$u\bar{u} \to d\bar{d}g$	**5.54**	5.43–5.64	**2.13**	2.10–2.16	*1.46°*	2.87 ms	*1.15°*	2.26 ms	**1.35**	1.20–1.51
$u\bar{u} \to u\bar{u}g$	**4.96**	4.89–5.04	**1.63**	1.59–1.67	**1.16**	0.10–2.22	**0.87**	0.86–0.88	**0.87**	0.78–0.95
$u\bar{u} \to e^-e^+g$	**3.85**	3.83–3.86	**1.61**	1.60–1.62	*1.07°*	1.01 ms	*0.85°*	0.805 ms	**0.84**	0.84–0.84
$u\bar{u} \to ggg$	**3.05**	2.98–3.12	**0.84**	0.81–0.86	*0.68°*	11.0 ms	*0.74°*	11.9 ms	**0.63**	0.59–0.67
$gg \to ggg$	**1.66**	1.66–1.66	**0.45**	0.45–0.45	*0.42°*	52.7 ms	*0.34°*	42.9 ms	**0.36**	0.35–0.37
$u\bar{u} \to e^-e^+gg$	**3.21**	3.20–3.23	**0.78**	0.78–0.78	*0.62°*	8.20 ms	*0.42°*	5.54 ms	**0.47**	0.44–0.49
$gg \to gggg$	**1.08**	1.07–1.09	**0.31**	0.30–0.31	*0.30°*	1 120 ms	*0.26°*	977 ms	**0.25**	0.24–0.26
$gg \to t\bar{t}gg$	**1.35**	1.34–1.37	**0.37**	0.36–0.37	*0.35°*	215 ms	*0.28°*	172 ms	**0.30**	0.29–0.31
$u\bar{u} \to gggg$	**1.74**	1.71–1.77	**0.43**	0.43–0.44	*0.47°*	230 ms	*0.37°*	178 ms	**0.35**	0.34–0.36

2.1 Where the time goes

Table 4: **Stage split**, ms per point, full colour, at the largest batch timed (Python-input $2 \to 4$: $B = 1$). Trees/Born/CT: the per-point inputs, timed in Python before G-int (history: trees, counterterm program, Born and counterterm colour projections, scalars); the build on `loop` forms them in the executor without a timer (—: the development campaign's scratch timers of `39824f9b` timed them). Untimed rest: the part of the call such a build does not time, its native trees, Born and counterterm inputs and the rest of the Python call. Numerator: seeds, fused factors and the dressing of the open-loop objects (the pure numerator evaluation). Scatter: closures into NinjaRS lanes. NinjaRS: the reduction without the masters; masters: the master integrals (canonical arguments, cache, oneloopmaster). Glue: point parameters, carrier conversion, assembly and, where the inputs are timed, the rest of the Python call. MadLoop (instrumented copy; its points less one, 3 for $gg \to gggg$): Born/counterterm HELAS calls, loop-numerator coefficients, reduction plus masters, residual (stability test, colour sums). Bar: shares, colours as the column heads.

Process		Trees, Born, CT	Numerator	Scatter	NinjaRS	Masters	Glue	Untimed rest	Total	Shares
$gg \to gg$	native, `loop`	—	0.818	0.0882	0.113	0.0357	0.0994	1.06	**2.21**	
	Python[hist]	128	0.851	0.0919	0.119	0.0381	2.96	—	**132**	
	MadLoop	0.0315	1.27	—	0.585		0.107	—	**1.99**	
$u\bar{u} \to d\bar{d}g$	native, `loop`	—	0.753	0.466	0.138	0.112	0.138	1.06	**2.67**	
	Python[hist]	28.1	0.749	0.462	0.137	0.111	2.66	—	**32.4**	
	MadLoop	0.0479	2.35	—	0.683		0.133	—	**3.22**	
$u\bar{u} \to u\bar{u}g$	native, `loop`	—	1.22	0.477	0.224	0.164	0.167	1.04	**3.30**	
	Python[hist]	48.5	1.23	0.479	0.232	0.171	3.99	—	**54.5**	
	MadLoop	0.137	6.44	—	1.15		0.814	—	**8.54**	

Process		Trees, Born, CT	Numerator	Scatter	NinjaRS	Masters	Glue	Untimed rest	Total	Shares
$u\bar{u} \to e^-e^+g$	native, `loop`	—	0.268	0.153	0.0274	0.0181	0.0943	0.233	**0.796**	
	MadLoop	0.0184	0.754	—	0.133		0.0460	—	**0.951**	
$u\bar{u} \to ggg$	native, `loop`	—	2.84	0.470	0.449	0.208	0.193	6.02	**10.2**	
	Python[hist]	168	2.92	0.476	0.459	0.225	5.75	—	**179**	
	MadLoop	0.346	18.5	—	2.54		2.87	—	**24.3**	
$gg \to ggg$	native, `loop`	—	11.5	0.521	2.00	0.487	0.281	30.6	**45.4**	
	Python[hist]	2 530	11.7	0.519	2.03	0.483	10.5	—	**2 560**	
	MadLoop	2.42	184	—	11.3		39.5	—	**237**	
$u\bar{u} \to e^-e^+gg$	native, `loop`	—	2.72	0.636	0.421	0.230	0.185	2.04	**6.23**	
	MadLoop	0.289	22.7	—	2.38		2.11	—	**27.5**	
$gg \to gggg$	native, `loop`	—	211	2.41	44.4	5.81	1.63	652	**913**	
	Python[hist]	97 300	3 750	4.08	38.6	5.81	137	—	**101 000**	
	MadLoop	88.9	18 600	—	209		17 700	—	**36 500**	
$gg \to t\bar{t}gg$	native, `loop`	—	45.4	0.933	20.2	6.35	1.50	108	**183**	
	Python[hist]	6 490	751	3.55	20.7	6.58	29.3	—	**7 310**	
	MadLoop	24.2	3 290	—	105		1 740	—	**5 160**	
$u\bar{u} \to gggg$	native, `loop`	—	47.4	1.80	9.17	2.91	0.904	108	**170**	
	Python[hist]	4 910	806	2.01	9.64	2.99	24.1	—	**5 760**	

Verdict of gate G1 (processes from $2 \to 3$; $2 \to 2$ without a gate).

- **Native inputs on `loop`** (G1 at `4e9b5a9e`) at $B = 128$: **pass** for $u\bar{u} \to u\bar{u}g$, $u\bar{u} \to e^-e^+g$, $u\bar{u} \to ggg$, $gg \to ggg$, $u\bar{u} \to e^-e^+gg$, $gg \to gggg$, $gg \to t\bar{t}gg$ and $u\bar{u} \to gggg$; **fail** for $u\bar{u} \to d\bar{d}g$ (×1.4, OpenLoops binding). At $B = 1$ with 16 lanes none passes (a one-point call pays a 16-lane chunk, Table 3); with one lane per call, in the development runs of `39824f9b` (history), $u\bar{u} \to ggg$, $gg \to ggg$, $u\bar{u} \to e^-e^+gg$, $gg \to gggg$, $gg \to t\bar{t}gg$ and $u\bar{u} \to gggg$ passed.
- **Python inputs** history (the engine before G-int, G1 at `1c971289`): failed every process it generated, at $B = 128$ at ×11–20 of the binding limit, the per-point Python inputs taking 87 %–99 % of its time (Table 4); at $B = 1$ at ×12–27, where $gg \to gggg$, $gg \to t\bar{t}gg$ and $u\bar{u} \to gggg$ were timed only. It did not generate $u\bar{u} \to e^-e^+g$ and $u\bar{u} \to e^-e^+gg$ (Furry zeros, WP3.5; Z fermion signs, WP3.2).
- **Fastest of all tools** (aspiration), native inputs at $B = 128$, within the uncertainty band: faster than every provider for $u\bar{u} \to e^-e^+g$, $u\bar{u} \to ggg$, $gg \to ggg$, $u\bar{u} \to e^-e^+gg$, $gg \to gggg$, $gg \to t\bar{t}gg$ and $u\bar{u} \to gggg$; at parity with the fastest for none; slower than the fastest (OpenLoops) for $u\bar{u} \to d\bar{d}g$ and $u\bar{u} \to u\bar{u}g$. NJet, not a matched provider, takes 4.58 s for its two-evaluation full-colour $gg \to gggg$ test (Table 13).
- Seven-leg rows ($u\bar{u} \to e^-e^+d\bar{d}g$, $u\bar{u} \to e^-e^+ggg$, $gg \to t\bar{t}e^-e^+g$, $u\bar{u} \to ggggg$, $gg \to ggggg$) are generated by the WP3.1 generator, on `loop` since G-int, and are added to G1 by the follow-on package `measure-g1-7leg`; $gg \to ggggg$ has no matched provider timing (addendum F, H): reported without a gate.

Table 5: **Levers**, measured on the rows of Table 4. Gains are measured where a column says so; otherwise they are the stage's share, an upper bound.

Lever	Evidence	Expected gain	Owner
Native per-point inputs (WP3.1 I3–I6: trees, Born, counterterm scalars in the executor)	Python inputs 87 %–99 % of the Python-input time at $B = 128$ (history)	measured, Python ÷ native inputs at equal B: ×3.7–25 at $B = 1$, ×12–56 at $B = 128$ ($2 \to 3$)	done: G-int, on `loop` at 4e9b5a9e
Binary64 point inputs (the providers' input; the protocol passes the stream's exact rationals)	paired run on the loaded artifact, $B = 128$: $u\bar{u} \to d\bar{d}g$ 1.68 against 2.29 ms, $u\bar{u} \to u\bar{u}g$ 2.94 against 3.39 ms	measured: ×1.37 and ×1.15 (not a G1 configuration)	protocol: the user's decision; executor input conversion
Compiled native trees (kernels as the tree engine's SymJIT planes, lanes batched)	untimed rest (the native trees, Born and counterterm inputs and the rest of the call; no trees timer on `loop`) 29 %–71 % of the time at $B = 128$; $gg \to gggg$ 71 %	up to the share; ×3.5 for the largest	Rust ports A/B (`native-dag`, HANDOFF §0.4)
Lanes by batch in binary64 (`plan_export.lanes_for`: 16 lanes also for one point)	$B = 1$: one 16-lane chunk per call	measured: ×2.4–4.1 at $B = 1$ (16 lanes ÷ 1 lane of 39824f9b)	executor policy (WP3.1 or wave 4)
Finite-only result path (sum the virtual natively, no per-family lane export, Python assembly only for checks and observables)	glue 0 %–12 % of the native-input time	up to the share (largest at $2 \to 3$ with quarks)	`plan_export`, executor (wave 4)

Lever	Evidence	Expected gain	Owner
Lane-major closures (no scatter, no second copy into the reducer's batch)	scatter 0 %–19 %	up to the share	executor, reducer adapter (wave 4)
AVX2 / AVX-512 kernels (runtime selection)	numerator 23 %–44 %	×1.0–1.4 on the dressing (gate G0)	wave 4 (WP4.6)
NinjaRS and masters: parallel reduction, transposed reduction, stage-2 sharing, a cheaper master cache key	NinjaRS + masters 5 %–15 %; masters 12 %–45 % of the reduction, oneloopmaster alone 2 %–12 %: the cache bookkeeping costs more than the integrals	one thread: up to the share; threads: DESIGN §5.6	NinjaRS requests (DESIGN §11.5)

The failing row, $u\bar{u} \to d\bar{d}g$, takes 2.67 ms at $B = 128$ against a limit of 1.97 ms (OpenLoops 1.52 ms × 1.30). Its glue and scatter are 0.604 ms; without them it would take 2.06 ms, 1.05 of the limit. Its points enter as exact rationals. In a paired run on the loaded artifact (one core, ten alternating rounds, $B = 128$; not a G1 configuration, and its exact-input time, 2.29 ms, is not G1′s) the same executor takes 1.68 ms on the binary64 text the providers read (×0.73; $u\bar{u} \to u\bar{u}g$: 2.94 against 3.39 ms, ×0.87), and the public `Runtime.evaluate` adds 10 % (2 %) to the executor call on the same decimal text. Whether G1 passes binary64 inputs is the user's decision; this campaign keeps the exact inputs of the measure-g1 protocol, and no verdict is formed from the paired run.

2.2 Leading colour

Table 6: **Leading colour** (LC), ms per point, same protocol as Table 2; in each cell the full-colour (FC) time of the same configuration and batch, and LC/FC. No provider ratio: the providers time full colour. LC uses the committed QCD colour continuation (`examples/loop/source_colour`); the V+jets rows are timed at LC with native inputs only. This campaign timed LC at $B = 128$; ° grey: the earlier run of `39824f9b` at $B = 1$ with its FC time (bitwise the same values).

Process	Python inputs[hist]				Native inputs, `loop`			
	$B = 1$		largest batch		$B = 1$		largest batch	
$gg \to gg$	*not timed*		*not timed*		*not timed*		**1.56**	FC 2.21 · 0.71
$u\bar{u} \to d\bar{d}g$	**40.5**	FC 40.6 · 1.00	**37.4**	FC 32.4 · 1.15	*5.25°*	FC 10.9 · 0.48	**1.89**	FC 2.67 · 0.71
$u\bar{u} \to u\bar{u}g$	*not timed*		*not timed*		*9.00°*	FC 19.1 · 0.47	**2.72**	FC 3.30 · 0.82
$u\bar{u} \to e^-e^+g$	*not timed*		*not timed*		*not timed*		**0.622**	FC 0.796 · 0.78
$u\bar{u} \to ggg$	**184**	FC 215 · 0.86	**171**	FC 179 · 0.96	*22.8°*	FC 50.6 · 0.45	**12.0**	FC 10.2 · 1.18
$gg \to ggg$	**2 350**	FC 2 760 · 0.85	**2 300** ($B = 32$)	FC 2 550 · 0.90	*91.1°*	FC 211 · 0.43	**44.7**	FC 45.4 · 0.98
$u\bar{u} \to e^-e^+gg$	*not timed*		*not timed*		*not timed*		**3.42**	FC 6.23 · 0.55
$gg \to gggg$	*not timed*		*not timed*		*2 070°*	FC 4 100 · 0.50	**1 030**	FC 913 · 1.13
$gg \to t\bar{t}gg$	*not timed*		*not timed*		*418°*	FC 840 · 0.50	**217**	FC 183 · 1.18
$u\bar{u} \to gggg$	*not timed*		*not timed*		*380°*	FC 860 · 0.44	**218**	FC 170 · 1.28

LC is cheaper in the loop stage (fewer families and skeletons), but not yet end to end: with native inputs at $B = 128$ it is slower than full colour for $u\bar{u} \to ggg$, $gg \to gggg$, $gg \to t\bar{t}gg$ and $u\bar{u} \to gggg$. There the untimed rest of the call, which holds the native trees, Born and counterterm inputs, is larger at LC ($u\bar{u} \to ggg$: 10.0 of 12.0 ms at LC, 6.02 of 10.2 ms at full colour; $gg \to gggg$: 931 of 1 030 ms at LC, 652 of 913 ms at full colour), as the development campaign's timers showed for the trees of the processes with many colour orderings; with Python inputs (history) the inputs dominated either way.

2.3 Throughput, model recalibration and schedule M

Table 7: **Kernel throughput and the design recipe** (native inputs, full colour, largest batch). Work: the executor's modelled complex multiply-adds of the chosen kernel forms per point (one lane); throughput: work × lanes over the dressing time (fused-factor builds excluded). Native loop: seeds, factors, dressing, scatter and reduction; outside: everything else of the call (trees, glue). Recipe: DESIGN §1.3, finite only, where the design has a row; measured ÷ recipe for the total and for the NinjaRS stage (masters included) against its model R.

Process	Work, MCMAC/pt	Dressing, ms	G CMAC/s	Native loop, ms	Outside, ms	Total, ms	Total ÷ recipe	NinjaRS ÷ R
$gg \to gg$	0.932	0.681	1.37	1.05	1.16	**2.21**	1.5–4.4	0.94–1.07
$u\bar{u} \to d\bar{d}g$	0.556	0.442	1.26	1.47	1.20	**2.67**	no row	—
$u\bar{u} \to u\bar{u}g$	1.09	0.861	1.26	2.08	1.21	**3.30**	no row	—
$u\bar{u} \to e^-e^+g$	0.168	0.130	1.29	0.466	0.330	**0.796**	no row	—
$u\bar{u} \to ggg$	3.18	2.38	1.34	3.97	6.21	**10.2**	no row	—
$gg \to ggg$	14.3	10.7	1.35	14.5	30.9	**45.4**	no row	—
$u\bar{u} \to e^-e^+gg$	2.89	2.19	1.32	4.01	2.22	**6.23**	no row	—
$gg \to gggg$	268	204	1.31	264	649	**913**	2.2–6.6	0.98–1.16
$gg \to t\bar{t}gg$	56.0	42.7	1.31	72.9	110	**183**	1.2–3.3	0.99–1.38
$u\bar{u} \to gggg$	56.7	43.0	1.32	61.2	109	**170**	no row	—

Recalibration. The kernels run 1.26–1.37 G complex MAC/s per core on the real plans, the rate gate G0 measured on its synthetic programs (1.29–1.37); the dressing term of the recipe holds. The NinjaRS stage, masters included, lands on the design's model R (measured ÷ R 0.94–1.38, Table 7). What the recipe misses is the **outer factor**: it assumed

trees, seeds and counterterms add 5–10 % (DESIGN §1.3), but with the current native inputs the total is 1.86–3.5 times dressing plus reduction at full colour (up to 11 at LC). Table 8 re-estimates every design row with these measured terms.

Table 8: **Design estimates recalibrated by G1** (`tools/loop_v2_costmodel.py --g1`), ms per point, finite only, one thread. Recipe: DESIGN Table 1a's finite-only band (outer factor 1.05–1.10). G1: the recipe's dressing term, the NinjaRS model R × 0.97–1.39 (the band of the first G1 campaign, a constant of the tool; this campaign measures 0.94–1.38, Table 7) and the measured outer factor of the row's class representative at $B = 128$ with native inputs (same colour where measured). Measured: G1's own time where the row was timed. Gate: 1.3× the best production provider of DESIGN Table 1b (seven-leg rows, protocol of 20 October), with the headroom gate ÷ upper estimate. The outer factor shrinks with compiled trees (Table 5); until then these replace the Table 1a targets.

Design row	Col.	Recipe	Outer	Class	G1 estimate	Measured	Gate	Headroom
$gg \to gg$	FC	0.502–1.52	2.30	$gg \to gg$	**1.09–3.32**	2.21	—	—
$gg \to gg$	LC	0.204–0.542	5.04	$gg \to gg$	**0.969–2.64**	1.56	—	—
$gg \to gggg$	FC	139–409	3.52	$gg \to gggg$	**463–1 380**	913	3 800	2.75
$gg \to gggg$	LC	55.7–146	10.66	$gg \to gggg$	**559–1 520**	1 030	—	—
$gg \to ggggg$	FC	2 710–7 920	3.52	$gg \to gggg$	**8 980–26 700**	—	—	—
$gg \to ggggg$	LC	1 070–2 830	10.66	$gg \to gggg$	**10 800–29 500**	—	—	—
$u\bar{u} \to gggg$	FC	827–2 630	2.99	$u\bar{u} \to gggg$	**2 330–7 470**	—	—	—
$u\bar{u} \to gggg$	LC	206–526	11.40	$u\bar{u} \to gggg$	**2 200–6 000**	—	—	—
$u\bar{u} \to e^-e^+ggg$	FC	24.5–68.1	1.86	$u\bar{u} \to e^-e^+gg$	**42.8–124**	—	320	2.58
$u\bar{u} \to e^-e^+ggg$	LC	3.34–8.31	4.37	$u\bar{u} \to e^-e^+gg$	**13.7–36.5**	—	—	—
$u\bar{u} \to e^-e^+d\bar{d}g$	FC	5.34–16.5	1.86	$u\bar{u} \to e^-e^+gg$	**9.32–30.3**	—	45.1	1.49
$u\bar{u} \to e^-e^+d\bar{d}g$	LC	1.19–3.14	4.37	$u\bar{u} \to e^-e^+gg$	**4.87–14.0**	—	—	—
$gg \to t\bar{t}gg$	FC	54.8–156	2.60	$gg \to t\bar{t}gg$	**134–396**	183	633	1.60
$gg \to t\bar{t}gg$	LC	14.6–40.1	10.45	$gg \to t\bar{t}gg$	**144–410**	217	—	—
$gg \to t\bar{t}e^-e^+g$	FC	27.4–76.1	2.60	$gg \to t\bar{t}gg$	**67.0–194**	—	553	2.85
$gg \to t\bar{t}e^-e^+g$	LC	4.06–11.0	10.45	$gg \to t\bar{t}gg$	**39.9–113**	—	—	—

Table 9: **Schedule M against B** (native inputs, one thread), pure-gluon classes, the only classes where M merges seed entries (DESIGN §5.6). The schedule-dependent stages are seeds, factors, dressing, scatter and the NinjaRS reduction ("the reduction included", §1.6 item 5); the trees, Born and counterterm inputs and the glue are the same work in both schedules. M: the development campaign's runs (`39824f9b`, not repeated); B: the configuration on `loop` (`4e9b5a9e`), on the same plans (bitwise the same binary64 values); LC was re-timed at $B = 128$ only. M is adopted where M/B ≤ 0.85 on the schedule-dependent stages at every common batch size; ± is the timing uncertainty of the ratio.

Process	Col.	Stages M/B, $B = 1$	Stages M/B, $B = 128$	End to end, $B = 1$	End to end, $B = 128$	Adopt M
$gg \to ggg$	FC	**1.00** ± 0.00	**1.02** ± 0.05	1.00	1.12	no
$gg \to ggg$	LC	—	**0.95** ± 0.04	—	1.04	no
$gg \to gggg$	FC	**0.90** ± 0.01	**0.89** ± 0.08	0.91	0.99	no
$gg \to gggg$	LC	—	**0.80** ± 0.02	—	0.99	**yes**

B against M (decision, provisional until the trees are compiled). On the schedule-dependent stages M passes the 15 % rule for $gg \to gggg$ at LC (0.80; DESIGN §5.6 measured 0.75–0.80 on the dressing side), and not for $gg \to ggg$ at FC (1.00–1.02), $gg \to ggg$ at LC (0.95) and $gg \to gggg$ at FC (0.89–0.90). The same rows qualify as in the development campaign's paired runs, on which the decision was made (`data/g1.json` of `c3a7be09`). End to end the gain is diluted by the tree inputs, which both schedules share and which compiled trees will shrink (Table 5). With a quark line M's entries do not merge at all (M/B work 1.00, DESIGN §5.6; equal by test). **Decision (G1):** schedule M for pure-gluon classes at leading colour from $n = 6$; schedule B for every other class at $n \leq 6$, including pure-gluon full colour; M stays the planned default from $n \geq 8$, and both are re-measured at seven legs with the WP3.1 plans (`measure-g1-7leg`).

Table 10: **Values at the timed points**: pyAmpliCol's binary64 finite part (native inputs on `loop`) against each provider's production-mode value on the common stream points (the providers read the binary64 decimal text of the exact points), largest deviation, metric $|a - b| \,/\, \max(|a|, |b|, |B|\ \alpha_s/2\pi)$; Recola's finite part converted with the universal double pole (it returns no poles in production). Worst point: the point of the row's largest deviation, evaluated again with one rotation and the reduction-direction check in binary64 and in double-double (DD) by the stability runs of `39824f9b`, not repeated (the timed binary64 values are bitwise theirs): E is the binary64 estimate, DD the DD value's deviation from the closest provider (named). Campaign: the first three points against the committed references (`tests/fixtures/loop/required`), fixture metric and tolerance, native inputs (`loop`) and Python inputs (history).

Process	Pts	vs MadLoop	vs Recola	vs OpenLoops	Worst point: E	DD: closest provider	Campaign, native	Campaign, Python[hist]
$gg \to gg$	16	1.7×10^{-12}	7.4×10^{-13}	7.5×10^{-13}	—	—	—	—
$u\bar{u} \to d\bar{d}g$	16	6.8×10^{-13}	3.3×10^{-10}	6.2×10^{-13}	pt 1: 7.2×10^{-12}	3.2×10^{-13} MadLoop	3.6×10^{-10}	3.6×10^{-10}
$u\bar{u} \to u\bar{u}g$	16	3.4×10^{-12}	7.2×10^{-11}	6.8×10^{-12}	pt 1: 9.2×10^{-12}	2.4×10^{-13} MadLoop	8.6×10^{-11}	8.6×10^{-11}
$u\bar{u} \to e^-e^+g$	16	8.8×10^{-14}	4.9×10^{-14}	no library	pt 0: 1.4×10^{-12}	7.5×10^{-15} Recola	—	—
$u\bar{u} \to ggg$	16	1.4×10^{-12}	1.5×10^{-12}	8.4×10^{-11}	pt 2: 1.2×10^{-12}	1.3×10^{-14} MadLoop	—	—
$gg \to ggg$	16	1.7×10^{-6}	1.7×10^{-6}	1.7×10^{-6}	pt 10: 9.9×10^{-6}	6.0×10^{-12} Recola	—	—

Process	Pts	vs MadLoop	vs Recola	vs OpenLoops	Worst point: E	DD: closest provider	Campaign, native	Campaign, Python^{hist}
$u\bar{u} \to e^-e^+gg$	16	1.1×10^{-10}	5.0×10^{-11}	no library	pt 8: 2.5×10^{-9}	4.3×10^{-13} Recola	—	—
$gg \to gggg$	8	1.7×10^{-7}	4.0×10^{-7}	4.0×10^{-7}	pt 1: 4.8×10^{-6}	4.4×10^{-12} Recola	4.0×10^{-7} **above 2.0×10^{-8}**	8.7×10^{-8} **above 2.0×10^{-8}**
$gg \to t\bar{t}gg$	16	7.6×10^{-9}	1.5×10^{-9}	1.5×10^{-9}	pt 13: 5.6×10^{-8}	5.5×10^{-15} OpenLoops	1.1×10^{-11}	1.1×10^{-11}
$u\bar{u} \to gggg$	16	—	9.4×10^{-12}	1.9×10^{-10}	pt 9: 2.7×10^{-9}	1.9×10^{-13} OpenLoops	—	—

Reading the values. The Born agrees with every provider to 1.0×10^{-13} or better. For the finite part, the double-double (DD) value at each worst point (Table 10) separates the sources: it agrees with the closest provider to 7.3×10^{-12} or better everywhere. Where pyAmpliCol's binary64 estimate E exceeds the deviation — $gg \to ggg$, $gg \to gggg$ and $gg \to t\bar{t}gg$ (above 10^{-9}) and $u\bar{u} \to gggg$ (1.9×10^{-13} in DD against 1.9×10^{-10} in binary64) — the deviation is pyAmpliCol's own binary64 loss, which E flags. The other deviations above 10^{-11} stay in DD and are the providers' own accuracy: Recola's at $u\bar{u} \to d\bar{d}g$ (3.3×10^{-10}), $u\bar{u} \to u\bar{u}g$ and $u\bar{u} \to e^-e^+gg$ (COLLIER's default target 10^{-8}, HANDOFF §7), OpenLoops' at $u\bar{u} \to ggg$ (8.4×10^{-11}) and MadLoop's at $u\bar{u} \to e^-e^+gg$ (1.1×10^{-10}); MadLoop's production values also stay off the DD values at the ill-conditioned gluon and top points (up to 2.4×10^{-7}). At the campaign points $gg \to gggg$ point 1 misses the fixture tolerance (2×10^{-8}) in binary64 on both paths, within its estimate $E = 4.8\times10^{-6}$; its DD value agrees with Recola to 4.4×10^{-12}.

3 Required processes: references and development validation

The plan requires the $2 \to 5$ processes of Table 1 (plan §5.7), compared with MadLoop and Recola, and with OpenLoops where a public library exists. A reference provider is run only if its generation completes in about 45 minutes, or in a time comparable to pyAmpliCol's own generation of the process (HANDOFF §0.1, addendum F). Otherwise pyAmpliCol is validated by Ward and gauge identities, Catani poles, LC/NLC/FC and DD/AP consistency, stability checks, lower multiplicities and existing references.

Table 11: **Provider generation on this host and the 45-minute rule** (HANDOFF §0.1): the rows of Table 1 where a provider generation takes more than 45 minutes or did not finish. MadLoop: graph generation + source export + compilation; Recola: process generation; OpenLoops: prebuilt public libraries (no generation time). Records: `data/g1.json` (G1 rows) and `data/measured.json` (`provider_generation`, the reference runs of the $2 \to 5$ rows), through `data/summary.json`. Every other row of Table 1 generates within 45 minutes with every provider (longest: MadLoop 10.9 min for $gg \to t\bar{t}e^-e^+g$ (EW)).

Process	MadLoop	Recola	OpenLoops	45-minute rule
$gg \to gggg$	2.4 h (285 + 8 354 + 37 s)	35.4 min	prebuilt ppjjjj	**MadLoop over 45 min; results exist and are used**
$u\bar{u} \to ggggg$	12.2 h (5 793 + 38 040 + 122 s)	2.8 h	no public library	**MadLoop, Recola over 45 min; results exist and are used**
$gg \to ggggg$	diagrams 17.4 h; unfinished (last state: exporting)	unfinished (last state: running, 2026-10-20)	no public library	**stopped 22 Oct; not to be restarted (HANDOFF section 0.1)**

Table 12: **Native validation of the required set** DEV (work/v2-generation-scale before its integration into loop at 4e9b5a9e, DESIGN WP3.1: ninth increment (6f5a64cc; 8 pinned cores, native stages rayon-parallel) for $u\bar{u} \to e^-e^+d\bar{d}g$, $gg \to gggg$, $gg \to ggggg$; eighth increment (0ea83af2; thread count not recorded) for $u\bar{u} \to e^-e^+ggg$, $gg \to t\bar{t}e^-e^+g$ (QCD), $gg \to t\bar{t}gg$; seventh increment (fef92189; one generation thread) for $u\bar{u} \to ggggg$): generation (LoopSlice.generate, then the plan export; peaks of each stage; shared host under load), then binary64 virtuals with native per-point inputs and the production counterterm route at the fixture points, against the references committed on that branch (tests/fixtures/loop/required; fixture tolerance 2×10^{-8}). Largest deviation over the references and points, metric of Table 1; the per-reference values are in data/measured.json. For $gg \to ggggg$ the references are NJet (double-double and two binary64 runs) and the Catani operator for the poles, top-free with $N_f = 5$; its estimate column is the rotation plus reduction-direction check per point.

Process	References	Born	Finite	$1/\varepsilon$	$1/\varepsilon^2$	Estimate E	Generation + export
$u\bar{u} \to e^-e^+d\bar{d}g$	ML, Rc, OL · 3 pts	1.9×10^{-14}	8.7×10^{-13}	2.3×10^{-13}	1.0×10^{-13}	1.2×10^{-12}, 2.8×10^{-12}, 1.1×10^{-12}	37.7 s 0.4 GiB
$u\bar{u} \to e^-e^+ggg$	ML, Rc, OL · 3 pts	2.3×10^{-14}	8.5×10^{-13}	4.6×10^{-13}	2.0×10^{-13}	—	36.7 s 0.5 GiB
$gg \to t\bar{t}e^-e^+g$ (QCD)	ML, Rc, OL · 3 pts	1.4×10^{-14}	5.7×10^{-9}	3.7×10^{-12}	5.3×10^{-12}	—	43.7 s 0.6 GiB
$gg \to t\bar{t}gg$	ML, Rc, OL · 3 pts	2.1×10^{-14}	1.1×10^{-11}	1.0×10^{-10}	1.6×10^{-11}	—	38.5 s 0.7 GiB
$gg \to gggg$	Rc, OL · 2 pts	4.7×10^{-14}	4.5×10^{-11}	1.2×10^{-11}	5.2×10^{-12}	1.5×10^{-10}, 1.4×10^{-10}	154 s 2.2 GiB
$u\bar{u} \to ggggg$	ML, Rc · 3 pts	1.7×10^{-13}	2.9×10^{-10}	6.1×10^{-10}	4.4×10^{-11}	—	517 s 6.1 GiB
$gg \to ggggg$	NJet DD, NJet R1, NJet R2, Catani · 1 pt	8.1×10^{-13}	3.8×10^{-9}	1.5×10^{-9}	1.1×10^{-12}	2.0×10^{-8}	51.5 min 28.7 GiB

The $gg \to t\bar{t}e^-e^+g$ (QCD) finite part deviates by 5.7×10^{-9} at one of its three points and by at most 3.7×10^{-12} at the others: the point's known binary64 conditioning (earlier record, DESIGN WP3.1 fourth increment: ≤ 3.6×10^{-12} at two points; third point 5.7×10^{-9} in binary64 (estimate 1.7×10^{-9}), MadLoop to 1.9×10^{-14} in DoubleFloat). The electroweak target is not started (WP3.2). The $u\bar{u} \to ggggg$ row comes from DESIGN.md WP3.1 seventh increment (fef92189), the newest record of that process: the later re-runs on the merged heads (eighth and ninth increments) do not include it. It uses the same production counterterm route; its check is in Table 14.

4 Seven gluons: NJet and pyAmpliCol

4.1 NJet: Table 5 of arXiv:1209.0100, re-measured on this host

NJet is the independent reference for $gg \to ggggg$, where MadLoop and Recola cannot generate under the 45-minute rule and OpenLoops has no library. It is massless five-flavour QCD, and it is not one of the gate's matched providers.

Table 13: **NJet, s per point**, double precision, two evaluations each (NJet's scaling test). Paper: Xeon E3-1240 at 3.3 GHz, NJet 1.0. This host: one pinned core, idle at launch, of the shared host; 10 frozen points per channel, mean (spread ≤ 2 %); the multiplier is time ÷ the paper's. De-symmetrised sums equal the standard sums only after phase-space integration; they are timing figures, not point values.

Channel	Table 5	NJet 3.1.2		NJet 2.1.1		Table 5	NJet 3.1.2		NJet 2.1.1	
	standard full-colour sum					de-symmetrised sum				
$gg \to gg$	0.03	**0.00295**	×0.1	**0.00291**	×0.1	0.03	*no class*		*no class*	
$gg \to ggg$	0.22	*defective channel*		**0.141**	×0.6	0.07	*defective channel*		**0.0452**	×0.6
$gg \to gggg$	6.19	**4.59**	×0.7	**4.58**	×0.7	0.57	**0.39**	×0.7	**0.38**	×0.7
$gg \to ggggg$	171.31	**142**	×0.8	**246.7**	×1.4	3.07	**2.41**	×0.8	**4.19**	×1.4

At seven gluons one NJet 3.1.2 evaluation without the scaling test takes 71.2 s (standard) and 1.22 s (de-symmetrised; means over 10 points), at 69 MB; double-double costs 1 990 s per point. NJet 3.1.2's numerical five-gluon channel is 9/2 too large (NJet 2.1.1 is used there), and at the unrotated seven-gluon points NJet keeps only 4–6 digits in double precision (NaN at 3 of 10), so the reference values are taken at exactly rotated points.

4.2 pyAmpliCol at seven gluons DEV

On the WP3.1 branch before its integration (G-int; Table 12) the full $gg \to ggggg$ virtual, with the counterterms of the recursion and native per-point inputs, top-free with $N_f = 5$, agrees with NJet's double-double values to 3.8×10^{-9} in the finite part (per point: 3.8×10^{-9}, 3.1×10^{-9}, 4.3×10^{-11} over all references, points 0, 1, 2, eighth increment) and to 5.3×10^{-12} in the double pole; each point's rotation plus reduction-direction estimate bounds its difference. The ninth increment re-ran point 0 with equal deviations; Table 1 keeps the more complete comparison. Generation (ninth increment) took 44.2 min at 14.7 GiB and the plan export 7.3 min at 28.7 GiB (peak RSS of each stage; 160 MB of plan JSON; the whole validation run, point evaluations included, peaked at 38.6 GiB), inside addendum H's budget (hours, below 150 GB). DESIGN WP3.1 attributes the cost: 325 s before the counterterm recursion, 1,122 s with its counterterm program, 2,766

s once poles-uvir's always-generated companion merged (eighth increment), and 2,652 s after the ct-simplify merge (ninth increment, this record), 1,843 s of it the companion's seed table, which builds generic seeds (making it lean is open). A point with its rotation and reduction-direction checks took 82.3 s, a single-point development evaluation (8 pinned cores, native stages rayon-parallel): not a batched one-thread time and not comparable with NJet's 71.2 s per evaluation on one pinned core. No batched seven-gluon timing exists yet.

4.3 Validation without a matched provider

Table 14: **Checks of the seven-leg QCD results**, DEV on `work/v2-generation-scale`, except the Ward row (WP2.5 on `loop`); binary64, all three Laurent coefficients, deviation metric of Table 1. NJet values: fixture `gg_ggggg_njet.json` (NJet 3.1.2 double-double at exactly rotated points, scaling-test estimates $\lesssim 1.3\times10^{-24}$).

Check	Result	Status
$gg \to ggggg$ full virtual against NJet (top-free, $N_f = 5$), 3 points (eighth increment; ninth: point 0)	Born $\leq 7.5\times10^{-13}$, finite $\leq 3.8\times10^{-9}$, single pole $\leq 2.4\times10^{-9}$, double pole $\leq 5.3\times10^{-12}$; Catani poles: single $\leq 2.4\times10^{-9}$, double $\leq 5.3\times10^{-12}$; estimates 2.0×10^{-8}, 6.0×10^{-8}, 1.4×10^{-10}	passed (tolerance 2×10^{-8})
Top-free full virtual against NJet at lower multiplicity	$gg \to gg$: $\leq 2.1\times10^{-14}$ at 10 points, full virtual; $gg \to ggg$: $\leq 2.3\times10^{-13}$ at the stable points; the flagged point 3×10^{-14} in DoubleFloat; $gg \to gggg$: 1.0×10^{-11} at point 0; point 1 1.6×10^{-7} in binary64 (estimate 2.7×10^{-6}), 4.7×10^{-15} in DoubleFloat (`test_loop_v2_topfree.py`)	passed
$u\bar{u} \to ggggg$ full virtual against MadLoop and Recola, 3 points (DESIGN.md WP3.1 seventh increment (fef92189))	MadLoop $\leq 2.6\times10^{-12}$, Recola $\leq 6.1\times10^{-10}$ (finite $\leq 2.9\times10^{-10}$); the providers agree to 6.2×10^{-10} (reference merge)	passed (tolerance 2×10^{-8})
Ward identity, one gluon's $\varepsilon \to p$, at seven gluons	asymmetric Ward mode of WP2.5 (loop side only, bare + UV + R_2); passes on $gg \to gg$ and $u\bar{u} \to ggg$ (`test_loop_v2_observables.py`)	**not yet run at seven gluons (addendum H)**

5 Capabilities at this milestone

Table 15: **Capability matrix** of the v2 engine on `loop` at 4e9b5a9e, with branch work marked.

Feature	Status	Boundary
Recursive engine	supported	Trees, families and loop numerators from recursions; no loop diagram is enumerated (FeynKit graphs are test oracles only). Counterterms: see below.
Full, NLC and LC colour	supported	Exact seed tables in N_c; LC/NLC by generated classes and truncated cells. LC is cheaper in the loop stage but not yet end to end at large batches (its native per-point inputs cost more, Table 6), nor to generate at seven legs.
Colour and spin correlations	supported	Directed colour histories at LC/NLC/FC, spin sources, resolved helicities, flavour aliases, Ward sources (WP2.5). Seven-leg directed LC/NLC tables need the Rust seed tables (WP3.1).
Per-order partials	supported	Every squared coupling order at tree level and at one loop, mixed Born orders, per-correction loop orders.
Poles at runtime	supported	`poles = "all" | "none" | "uv" | "ir" | "split"` (WP2.10, on `loop` since 24 October): finite-only mode with the finite part bitwise unchanged and the pole work skipped; `"uv"` and `"ir"` with the user's semantics of 23 October. The UV residue comes from an interim local module until the NinjaRS re-pin; a `"uv"` estimate needs a rotation check.
Stability estimates	supported	Rotation checks (any number, 0 = estimate absent), reduction direction, exact rescaling, in any combination; no automatic escalation (WP2.3).
Precision carriers	supported	Binary64, DoubleFloat, arbitrary precision and exact rationals in the native executor; since G-int the native per-point inputs run in binary64, DoubleFloat and arbitrary precision (DESIGN §5.5).
Native executor, schedules	supported	Rust, 16 lanes (also for a one-point call, a G1 lever), threads; G1 decision on the schedule-dependent stages (Table 9, provisional until compiled trees): M for pure-gluon LC from $n = 6$, B for every other class at $n \leq 6$; F as oracle. The NinjaRS reduction runs serially. Development report: stage times, modelled kernel work, master-integral time.
Plan v4 artifacts and API	partial	Published and loaded through Python, TOML and CLI; relocation to another path tested on the same host and CPU. Since G-int the artifact stores the native process (`point_inputs = "native-v1"`): a plain evaluation loads without any generation call. Correlated, resolved and flavour calls and the `"uv"`/`"ir"` selections still regenerate the slice on first use, and processes the native export refuses keep `regenerated-at-load` (addendum G; generation-scale-2). The Rust/C/C++/Fortran SDK refuses v4 until then.
Counterterms (model UV, R_2)	supported	Plain tree vertices of the recursion inserted exactly once (WP2.2, a18b7599; addendum J, b4d76cc3), admitted by coupling orders and the public `particle_veto`; no witness table or loop matching in production. An unlowerable counterterm vertex that no tree of the request reaches is inert and recorded in the artifact; a reachable one is refused. Site closure is the test oracle. Per point they are formed natively (G-int), the Python stages kept as oracle; the native counterterm-program builder (A3) is on `loop`, not switched on.
Direct R_2	deferred	A bonus by the user's rule; bounded controls only.
Electroweak, complex mass	open	Tree EW lanes fixed, the Dirac–Weyl class refused; loop EW, Goldstones and CMS are WP3.2.
Required set	in progress	Seven of eight generate and agree with their references in all coefficients with native inputs (Table 12, validated on the WP3.1 branch before its integration), $gg \to ggggg$ against NJet's double-double values; the generator is on `loop` since G-int. The EW target is not started; no batched $2 \to 5$ timing.

Feature	Status	Boundary
Performance gates	**in progress**	G1 on `loop` (Section 2, native per-point inputs, 4e9b5a9e) at $B = 128$: 8 of 9 pass, $u\bar{u} \to d\bar{d}g$ fails; $B = 1$ needs lane adaptation (Table 3). The Python per-point inputs before G-int failed every gate they generated (history). Missing: provider protocol-mode columns, seven legs (`measure-g1-7leg`).
$gg \to 6g$	**deferred**	No $2 \to 6$ work until the $2 \to 5$ plan is complete (user, 23 October).

6 Architecture of the v2 engine

6.1 The computed quantity

For Born order a, loop order b and squared coupling order $s = a + b$,

$$V_{s,k} = \mathcal{N} \sum_{a+b=s} 2\,\mathrm{Re}\big[B_a^\dagger G_{BL} L_{b,k}\big], \quad k \in \{0,-1,-2\}, \quad V = C_\varepsilon \left(V_0 + \frac{V_{-1}}{\varepsilon} + \frac{V_{-2}}{\varepsilon^2}\right), \quad C_\varepsilon = \frac{(4\pi)^\varepsilon}{\Gamma(1-\varepsilon)},$$

where G_{BL} is the colour metric between Born and loop colour structures and $\mathcal{N}$ holds the initial-state averages and identical-particle factors (MadLoop/BLHA convention; couplings included, nothing divided by the Born). Directed colour and spin correlations replace $2\,\mathrm{Re}(\ldots)$ by the two cross terms $B_A^\dagger G L_B + L_A^\dagger G B_B$.

6.2 Pipeline

For every denominator family f, sector s and observable column o, v2 builds the four-dimensional numerator $N_{f,s,o}(q) = \sum_\alpha c_\alpha q^\alpha$ that NinjaRS reduces: Born-weighted and helicity-summed, produced by three recursions that share all work (Figure 2).

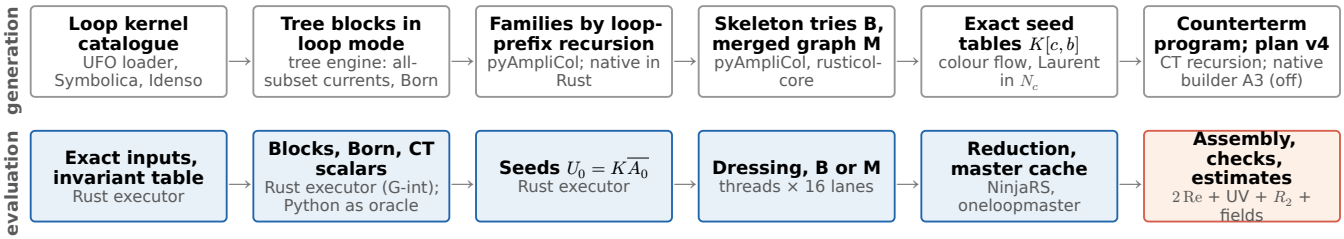


Figure 2: **v2 pipeline.** Upper lane: generation, once per process, into a plan v4 artifact. Lower lane: evaluation per batch of points. Blue: native batched numerics in the Rust executor (the per-point inputs since G-int); orange: Python per batch; white: generation. The owning code or dependency is named under each step.

Trees. The tree engine's colour-ordered Berends–Giele recursion computes, once per point, every off-shell current over every leg subset a loop vertex can absorb (the *blocks*), the Born colour components A_0 and, with WP2.2, every tree with one counterterm vertex. A loop-aware reachability test keeps exactly the currents some family position or root needs. Blocks with transverse sources use the exact transverse Yang–Mills kernels; loop segments, counterterm currents and Ward sources use the full ones.

Families and skeletons. A Recola-type recursion over loop prefixes (ordered leg sets, propagator species and masses) yields the denominator families, keyed by the canonical cyclic sequence of (leg mask, mass) and equal to Recola's tensor-integral identities. Per family it yields the colour-ordered *skeletons*: rings of blocks, each block summing all trees on its leg subset (Figure 3). Skeletons are never stored as a list; they are the leaves of the schedule tries. Only certified structural zeros are dropped (scaleless rings, external wave-function corrections, VEV tadpoles, colour zeros); nothing is restricted to 1PI.

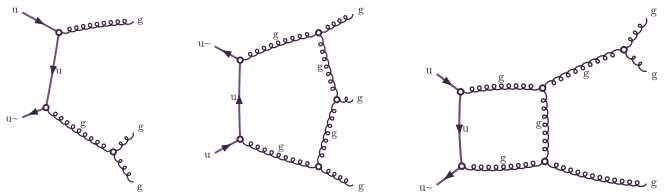


Figure 3: Three graphs of $u\bar{u} \to ggg$ drawn by FeynKit's renderer from its own generation (an oracle, not a production input). In v2 the Born (left) is a closure of blocks; the pentagon (middle) is a ring of five single-leg blocks; the box with a tree attachment (right) is a ring of four blocks, one of them the two-leg current, computed once per point and shared by every skeleton that contains that leg subset.

Loop numerators and schedules. An open-loop recursion dresses skeletons one segment at a time with polynomial-valued objects, $O' = O \cdot Y + \mathrm{raise}\big(O \cdot Z_\mu, q^\mu\big)$, in the NinjaRS monomial layout ($\binom{r+4}{4}$ coefficients at rank r, no rank

cap). The seed is the colour-summed Born interference, and each block's helicities are summed as it is attached. One generator feeds three schedules (Table 16).

Table 16: **Three schedules on one generator** (DESIGN §2.2, §5.6).

	F: forward	B: suffix trie (default)	M: merged, primitive-seeded
Born weighting	at closure	at the leaves (one seed per skeleton class)	at the cut, one seed per colour ordering
Shared	prefixes across families	all prefixes with equal family and remainder	prefixes across families and compositions
Colour certificate	none needed	none needed	certified classes: adjoint rings, traces, quark rings
Role	oracle	default; generic fallback	pure-gluon LC from $n = 6$ (G1); planned default from $n \geq 8$

Colour. Colour lives only in exact seed tables $K[c, b]$, Laurent polynomials in N_c between seed classes c (the colour vector obtained by closing the cut of a skeleton's colour flow) and Born basis elements b; per point the seeds are $U_0 = \sum_b K[c, b]\overline{A_0[b]}$, with a symmetric-group FFT where certified. LC and NLC differ from full colour only in which classes are generated and how cells are truncated: at LC each colour ordering is seeded by its own Born amplitude and skeletons below the leading N_c degree are never generated. Certificates (adjoint rings, fundamental traces, quark rings) are proved at loop level, never inherited from the tree level.

Reduction and masters. Families close directly into NinjaRS `TensorNumerator` lanes: one reduction per (family, sector, column), families with equal (propagators, rank) batched as lanes of one call, scalar masters from oneloopmaster cached per event. A second reduction in a relabelled frame reuses the numerators and the cached masters as the cheap stability check.

Counterterms. The model's UV and R_2 vertices, two-point ones included, are ordinary vertices of the tree recursion, and each contribution contains exactly one of them; external field factors are separate sources. Admission is the requested coupling orders, the exactly-once constraint and a public `particle_veto`: a coupling term whose loop particles include a vetoed particle is dropped, and a vetoed particle is removed from every tree vertex, loop line and propagator of the generation, so loops and counterterms stay consistent. Nothing is matched to the loop (user, 23 October; addendum J). On `loop` the counterterms come from the WP2.2 recursion (one counterterm order in the tree recursion, the default route, whose $gg \to ggggg$ program leaves no rule to site closure) with this admission (`ct-simplify`, b4d76cc3): no witness table and no loop matching. A counterterm vertex that the model cannot lower and that no tree of the request reaches (the tree engine's own reachability, with the program's order limits and `particle_veto`) is inert and recorded in the artifact; a reachable one is refused. `particle_veto = [t]` replaces the internal top-free switch of Table 14; site closure and the witness machinery remain only as test oracles.

Precision, stability and assembly. Every kernel is generic over binary64, DoubleFloat, arbitrary-precision floats and exact rationals. A failed lane fails its point and never branches a kernel. The assembly takes $2\,\mathrm{Re}$ of $i/(16\pi^2)$ times the reduced family sum, plus local UV, R_2 and field terms, per sector and column, with optional rotation, reduction-direction and rescaling estimates and the finite-only mode. The public API selects `loop_engine = "v2"` from TOML, CLI or Python and stores plan v4 as a relocatable PACBIN member (zlib JSON of exact kernel term lists, no compiled code, digest checked at load) and, since G-int, the native process (`point_inputs = "native-v1"`: the tree program with the Born, counterterm and field components and their saved exact evaluators), so a plain evaluation loads without regenerating the process.

Table 17: **Asymptotic classes per event** for n gluons, full colour (DESIGN §1.4): $F_n \approx (n-1)!\left(\frac{1}{\ln 2}\right)^n/2$ families, 2^n helicities. The required set ($n \leq 7$) is far below the crossover of schedule B with the unitarity class ($n \approx 30$–50); schedule M is planned as the default from $n \geq 8$.

Stage or code	Class
Tree blocks	$\Theta(n \cdot n!)$ currents; $\Theta(n! \cdot 2^n)$ current-helicity rows
Dressing, schedule B	$\Theta((n-1)! \cdot 4^n)$ seed injection; trie dressing dominates for $n \leq 8$
Dressing, schedule M	$\Theta(n! \cdot 2^n \cdot n^4)$
NinjaRS	$\Theta(F_n)$ reductions, $\Theta(F_n \cdot n^8)$ work, $\Theta(4^n)$ distinct masters after caching
MadLoop (per diagram, per helicity)	$\Theta(D_n \cdot 2^n \cdot n^4)$, D_n super-exponential ($gg \to gggg$: 26 975 loop diagrams)
Unitarity with exact helicity sums (NJet)	$\Theta((n-1)! \cdot 2^n \cdot n^{6\ldots9})$

Table 18: **Ownership of the mathematics.** pyAmpliCol reuses its dependencies for algebra, tensors, graphs and integrals and owns the recursions and the assembly.

Component	Responsibility
ufo_model_loader	Lossless UFO loop-model import: Laurent-valued parameters and couplings, UV/R_2 categories, loop groups
Symbolica	Expressions, coefficient extraction in q, evaluators; exact rationals and arbitrary precision
Spenso, Idenso (Community)	Lorentz and colour tensors behind the kernel catalogue and the colour metrics
FeynKit (Community)	Test oracles: graph and family counts, overall factors, drawings (Figure 3)
NinjaRS, oneloopmaster	Integrand reduction with Laurent expansion; scalar masters in all carriers
pyAmpliCol tree engine	Blocks, Born, coupling orders and colour flow (Python lane; Rust recurrence)
pyAmpliCol loop v2	Families, skeletons, schedules, seed tables, counterterm program, plan v4, native executor, API

7 Sources and reproducibility

- **Build:** `typst compile benchmarks.typ benchmarks.pdf` in `docs/performance_reports/loop/` (Typst 0.15).
- **Status, in-flight work, decisions:** `data/milestone.json`, from the `Integrate …` merges of 4e9b5a9e since c3a7be09 (A3 `c29d5899`, G-int `4e9b5a9e`); `workspace/reports/engine-v2/slice-status.md` (integration blocks, DECISIONS 2026-10-23/24; the user's gate decision of 2026-10-24, 16:05 UTC, is recorded in `data/milestone.json` `gate_decisions` and no longer applies after the re-measure); `HANDOFF_24_10_2026.md` §6; MASTER_PLAN 2026-10-23/24 entries; REQUIREMENTS_ADDENDUM A, D, F, G, H, I, J; DESIGN §5.5, §5.7 (G1), §6.1, §6.2, §12.4, WP3.1; the 24 October pushes (59badc7a and b4d76cc3, gates in slice-status.md).
- **Table 1, Figure 1:** `data/summary.json`, written by `scripts/build_summary.py` from `data/g1.json`, `data/measured.json`, `data/milestone.json` and `tools/loop_benchmark/processes.json` by its main-configuration rule (the most recent measured configuration of each cell; a newer check that only repeats part of an older comparison does not replace it; revision ancestry from `data/milestone.json`); every entry names its configuration and source (README).
- **Table 11:** HANDOFF §0.1 (addendum F); the provider generation records of Table 1 (`data/summary.json`): `data/g1.json` for the G1 rows and the reference runs' records (`workspace/reference/runs/required/<process>/`, collected into `data/measured.json`) for the $2 \to 5$ rows; MadLoop's production-build times of the $2 \to 5$ rows: the same runs' `madloop-production-result.json`. **Table 12, Section 3, Section 4:** the native validations on `work/v2-generation-scale` (DESIGN.md WP3.1 ninth increment (6f5a64cc) for $u\bar{u} \to e^-e^+d\bar{d}g$, $gg \to ggggg$; DESIGN.md WP3.1 eighth increment (0ea83af2) for $u\bar{u} \to e^-e^+ggg$, $gg \to t\bar{t}e^-e^+g$, $gg \to ggggg$; DESIGN.md WP3.1 seventh increment (fef92189) for $u\bar{u} \to ggggg$), records `workspace/tmp/generation-scale/gint2/after/`, `gint/val1/` and `i7/val/` (the earlier seven-gluon points: `gint/val1/`), collected into `data/measured.json`; the earlier $gg \to t\bar{t}e^-e^+g$ record: DESIGN WP3.1 fourth increment. **Conventions:** MadLoop standalone, Recola, OpenLoops 2.
- **Section 2 (Table 2 to Table 10):** `data/g1.json` and the per-point values `data/g1-values.json`, written by `python -m tools.loop_benchmark gate` (README and docstrings of `tools/loop_benchmark`); streams `tools/loop_benchmark/streams/`; run records archived in `workspace/reports/engine-v2/measure-g1/records/` (first campaign `runs/`, fix round `runs2/`, the re-measure on `loop` at `4e9b5a9e` `gint/`: its runs, logs, the stored-artifact confirmation and the paired binary64-input runs `confirm/<process>/ab.json`; provider records reused from the first campaign and the fix round, not re-run), MadLoop exports of the G1-only processes in `workspace/reference/runs/g1/`; recalibration `tools/loop_v2_costmodel.py --g1 data/g1.json`. Builds: `loop-native-v2-slice-4e9b5a9e` (`loop`, `4e9b5a9e`); history: `loop-native-measure-g1-genscale-39824f9b` (`work/v2-generation-scale` `889af02c` plus the timers of scratch commit `39824f9b`, tag `g1-native-inputs-39824f9b`: earlier runs, one lane, schedule M, stability) and `loop-native-measure-g1-033edf15` (Python inputs), release with one codegen unit and thin LTO; the harness is this branch's `tools/loop_benchmark`; the generation of 39824f9b quoted in Section 1: `data/g1.json` of `c3a7be09`. Schedule M at seven legs: DESIGN §5.6.
- **Table 13** and NJet prose: `workspace/reports/engine-v2/njet-benchmark.md` §2, §5, §6 (raw `workspace/reference/runs/njet/timing/`; the seven-gluon single evaluations are read from those records into `data/measured.json`); NJet, arXiv:1209.0100 Table 5.
- **Table 14:** `data/measured.json` (`scripts/collect_milestone.py <workspace>`); top-free values at lower multiplicity: DESIGN WP3.1 and `test_loop_v2_topfree.py`; the seven-gluon generation history: DESIGN WP3.1 eighth and ninth increments. Memory in GiB = 2^{30} bytes.
- **Table 15, Section 6:** `docs/development/loop_v2/DESIGN.md` §1–§3, §5.5, §5.6, §6, §10, §12, REQUIREMENTS_ADDENDUM J (counterterm rule) and the integration records above.
- **History:** the legacy per-diagram engine's snapshot of 2026-10-18 (`data/measurements.json`, `provider-cold.json`, `legacy-labels.json`, `colour-measurements.json`) stays for the DESIGN citations and is no longer shown; its table is in the earlier report revisions (README).